

Color Clash — Technical Requirements Document

Rebuild v2.0 · React PWA client · TypeScript authoritative services

Dhoola · Geeks for Games

18 September 2026

Contents

1. Document control	2
1.1 Scope	2
1.2 Non-negotiable architectural rules	2
2. Technology stack	3
2.1 Client	3
2.2 Services	3
2.3 Deliberate exclusions	3
3. Repository layout	4
4. The engine contract	4
4.1 Core reducer	4
4.2 Invariants asserted after every accepted command	4
4.3 Domain types	5
4.4 Commands	5
4.5 Resolutions	5
4.6 Events	5
4.7 RNG	5
5. Client architecture	5
5.1 Layering	6
5.2 The presentation boundary	6
5.3 Local persistence	6
5.4 Performance budgets	6
6. Network protocol	7
6.1 Transport	7
6.2 Connection lifecycle	7
6.3 Client → server messages	7
6.4 Server → client messages	8
6.5 Command acknowledgement — corrected semantics	8
6.6 Idempotency, sequencing and resync	8
6.7 Rate limits	8
6.8 Command handling — bots	9
7. Match service	9
7.1 Room model	9
7.2 Authorisation matrix	9
7.3 Turn clock	9

7.4 Bot scheduling	10
7.5 Match persistence and recovery	10
7.6 Graceful shutdown	10
8. Client reconnection	10
9. Configuration and environment	11
10. Security requirements	11
11. Testing strategy	12
11.1 Layers and ownership	12
11.2 Must-pass edge cases (carried forward, all still required)	12
11.3 Determinism harness	12
11.4 CI gates	12
11.5 Coverage expectations	13
12. Build and release pipeline	13
12.1 Build targets	13
12.2 Android release	13
12.3 Service deployment	14
13. Observability	14
14. Migration from v1.0	14
15. Definition of done	15

1. Document control

Field	Value
Document	Technical Requirements Document (TRD)
Product	Color Clash
Release	v2.0 — “Rebuild”
Status	Draft for approval
Depends on	PRD v2.0
Companion documents	UI/UX Design Document, System Architecture & Backend Schema

1.1 Scope

This document specifies **how** Color Clash v2.0 is built: languages, packages, contracts, protocols, budgets, tests and pipelines. Service topology, data storage and schema live in the System Architecture & Backend Schema document; screen design lives in the UI/UX document.

1.2 Non-negotiable architectural rules

These are structural, enforced by lint rules and by CI, not by convention.

1. **The engine is pure.** packages/game-engine imports nothing outside @colorclash/shared and @colorclash/game-content. No React, no DOM, no network, no clocks, no Math.random, no globals.
2. **The UI never mutates game state.** It renders a PlayerView and dispatches Commands. A rule effect must never live in a click handler.
3. **The server is authoritative in online play.** Clients never decide legality.
4. **Bots submit the same commands as humans**, through the same validation pipeline, from the same PlayerView.
5. **Every ruleset registers through VersionManifest.** No version-specific branching outside a manifest, except where the source rule is itself version-scoped.

6. **All randomisation takes an injectable seeded RNG.** No exceptions — including bot id generation, which v1.0 got wrong.
7. **No silent rule invention.** An ambiguity becomes a named flag with a test and a `RULE_FLAGS.md` entry.

2. Technology stack

2.1 Client

Concern	Choice	Notes
Language	TypeScript 5.7+, <code>strict: true</code> , <code>verbatimModuleSyntax: true</code>	<code>import type</code> mandatory
UI	React 18 (function components, hooks)	No class components
Build	Vite 6	One bundler for all three targets — carried forward from v1.0's consolidation
Routing	react-router v6 with real URLs	Change from v1.0 , which used a single <code>useState<Screen></code> and had no URL state. Deep links (PRD DIST-04) require it.
Client state	Zustand stores + React Query for server state	Replaces v1.0's prop-drilling from a single 22-dependency <code>useMemo</code>
Styling	CSS custom properties + CSS modules	No CSS-in-JS runtime; the palette is defined once and mirrored from <code>game-content</code>
Realtime	Native WebSocket with a thin reconnecting wrapper	
Offline	Workbox-generated service worker, precaching exactly what was emitted	
Native shell	Capacitor 6, Android only	<code>appId: com.geeksforgames.colorclash</code>
Audio	WebAudio, synthesised at play time	Zero audio files ship

2.2 Services

Concern	Choice	Notes
Runtime	Node.js 22 LTS, TypeScript, ESM	An <code>engines</code> field is required in v2.0; v1.0 declared none
HTTP	Fastify	Replaces v1.0's hand-rolled <code>node:http</code> handler with no endpoints
WebSocket	ws behind the existing <code>WsConnection</code> interface	Replaces the hand-rolled RFC 6455 implementation (no frame cap, no backpressure, no heartbeat, no <code>Origin</code> check)
Validation	Zod at every trust boundary	Every inbound message and every <code>env var</code>
Auth	OIDC / JWT access tokens + rotating refresh tokens	
Database	PostgreSQL 16	
Cache / coordination	Redis 7	Presence, matchmaking queues, room registry, rate limits, pub/sub
Migrations	A versioned, forward-only migration tool checked into the repo	
Observability	OpenTelemetry traces + metrics, structured JSON logs	

2.3 Deliberate exclusions

- **No GraphQL.** The realtime surface is a WebSocket protocol; the rest is a small REST surface.
- **No ORM with implicit lazy loading.** A query builder or a thin typed client only, so every query is visible.
- **No serverless for the match service.** Matches are stateful and long-lived; they need a process that stays up.
- **No client-side rule evaluation in online play**, even as an optimisation.

3. Repository layout

/apps		
/client	React PWA – screens, animation, input, presentation	
/gateway	Fastify: REST API, auth, static, WS upgrade + routing	
/match	Authoritative match service (rooms, engine, bots, timers)	
/worker	Scheduled jobs: rollups, leaderboards, seasons, cleanup	
/packages		
/game-engine	Pure rules, state transitions, validators	[PORTED UNCHANGED]
/game-content	Version manifests, decks, colours, labels	[PORTED UNCHANGED]
/ai	Bot strategies over legal-move queries	[PORTED UNCHANGED]
/shared	Domain types, protocol schemas, RNG, utils	[PORTED + EXTENDED]
/test-fixtures	Deterministic decks and state snapshots	[PORTED UNCHANGED]
/db	Schema, migrations, typed data access	
/telemetry	Analytics provider abstraction	
/tools	Build packaging, asset generation, brand audit, test runner	
/docs	PRD, TRD, UI/UX, Architecture, RULE_FLAGS, BRANDING, PUBLISHING	

TRD-3.1 — npm workspaces, scope @colorclash/*, internal packages declared with explicit file: paths rather than "*" ranges, so resolution does not depend on the npm major version. tools/link-workspaces.mjs is retained as the documented fallback when npm mis-links on Windows.

TRD-3.2 — A dependency-boundary lint rule fails the build if game-engine, game-content, ai or shared import from apps/*, from React, or from any Node built-in that implies I/O or time.

4. The engine contract

Ported from v1.0 **unmodified**. Restated here because everything else codes against it.

4.1 Core reducer

```
reduce(state: GameState, command: Command, rng: RngLike):
  { state: GameState; events: GameEvent[] }
```

Pure. Deterministic. The same function backs local play, the authoritative match service, the bots and every test.

Pipeline inside reduce:

1. settleMissedClash(state, command, rng, events) — runs at the head of every command.
2. assertCommandIsAllowed(state, command).
3. Clone state; dispatch on command.type.
4. A ruleset never mutates state; resolveCard returns an ordered Resolution[] which applyResolutions applies.
5. checkTerminalConditions.
6. assertInvariants.

4.2 Invariants asserted after every accepted command

- Every card exists in exactly one zone.
- No player holds a duplicate CardId.
- Exactly one legal active player while PLAYING.
- activeColor is resolved once a wild settles.
- Recycling never moves the current top discard into the draw pile.
- Eliminated players can neither act nor be targeted.

- No stale pending choice.

TRD-4.2.1 — Invariant assertion is **on in production**. A violation throws, the command is rejected, and a `desync_detected` event is emitted. Failing loudly on a rule bug is cheaper than serving a corrupted table.

4.3 Domain types

`GameState` carries, in addition to the obvious zones and indices: `penalty: PenaltyContext` (`pendingDraw`, `minimum-ResponsePenalty`, `sourceKind`), `clashPending: PlayerId | null`, `clashCalled: PlayerId[]`, `mayPass`, `drawn-ThisTurn`, `seq`, `config: MatchConfig`, `seed`, `cards` (immutable after setup), and `stats`.

`stats.startedAt` / `stats.endedAt` are wall-clock and are the only fields of `GameState` that are not a pure function of the seed and the command log. Anything comparing two runs for determinism must exclude them; `statsFreeState` does that and `determinism.test.ts` pins both halves of the claim.

4.4 Commands

`START_GAME` · `PLAY_CARD` · `DRAW_CARD` · `CALL_CLASH` · `CHALLENGE_WILD_DRAW_FOUR` · `CHOOSE_COLOR` · `CHOOSE_TARGET` · `USE_FLEX` · `SWAP_HAND` · `END_TURN`

Every command carries `commandId` (idempotency key) and `playerId` (authorisation subject).

4.5 Resolutions

`SET_COLOR` · `REQUEST_COLOR_CHOICE` · `REQUEST_TARGET_CHOICE` · `REQUEST_SWAP_CHOICE` · `REQUEST_DRAW_COLOR_CHOICE` · `REVERSE` · `SKIP` · `NO_ADVANCE` · `DRAW` · `STACK` · `FLIP_DECK` · `SKIP_EVERYONE` · `ROTATE_HANDS` · `SWAP_HANDS` · `DISCARD_ALL_OF_COLOR` · `DRAW_UNTIL_COLOR` · `CONSUME_FLEX`

When a resolution raises a choice, everything after it in the queue is captured as a **continuation** on the `ChoiceState`, together with the accumulated advance re-encoded as resolutions. This is why a disconnect mid-choice cannot lose the rest of a card, and it must be preserved exactly.

4.6 Events

`GAME_STARTED` · `CARD_DEALT` · `CARD_PLAYED` · `COLOR_CHOSEN` · `TARGET_CHOSEN` · `CARD_DRAWN` · `STACK_UPDATED` · `TURN_CHANGED` · `CLASH_PENDING` · `CLASH_CALLED` · `CLASH_PENALTY` · `FLIP_TRIGGERED` · `FLEX_USED` · `PLAYER_ELIMINATED` · `HANDS_ROTATED` · `HANDS_SWAPPED` · `DISCARD_ALL` · `DECK_RECYCLED` · `CHOICE_REQUESTED` · `ROUND_ENDED` · `MATCH_ENDED`

4.7 RNG

```
type RngLike = {
  next(): number;           // uniform [0,1)
  int(maxExclusive): number;
  state(): number;          // serialisable cursor
};
```

TRD-4.7.1 — The RNG cursor is part of persisted match state. v1.0 omitted it, so a checkpoint restore re-ran `start()` (consuming the deal) and then jumped to a later checkpoint without advancing the cursor; any post-checkpoint reshuffle diverged. **This is a correctness defect being fixed, and it has a regression test: persist at a checkpoint, restore, force a draw-pile exhaustion, and assert the reshuffle matches the uninterrupted run.**

5. Client architecture

5.1 Layering

routes/	URL ↔ screen, guards, deep links
screens/	One component per screen; composition only
components/	Presentational, no game logic
hooks/	useMatch (local online), useProfile, useStore, useMatchmaking
state/	Zustand stores: session, profile, settings, match presentation
transport/	WebSocket client, REST client, retry and auth refresh
presentation/	Event → animation queue, banners, audio, haptics, pacing

TRD-5.1.1 — useMatch exposes an identical surface for local and online play (view, banner, log, reason, dispatch, clock). The table screen must be unable to tell which it is driving. This is v1.0's most valuable structural decision and is preserved.

5.2 The presentation boundary

PlayerView is both the local engine projection and the wire type. The client's only job on top of it is **pacing**: events arrive in a batch and are played out on a queue so hands tick down one seat at a time.

Constant	Value	Purpose
EVENT_CADENCE_MS	600–900	Time between presented events
FLIGHT_MS	260	Card travel animation
FLIGHT_MS_REDUCED	0	Reduced motion
BOT_PACE_MIN/MAX_MS	600 / 900	Bot decision pacing, local play
BOT_PACE_REDUCED_MS	200	Reduced motion
CLASH_GRACE_MS	4000	Floors the pace while a human owes a Clash call
REJECT_TOAST_MS	1800	How long a rejection reason is shown

TRD-5.2.1 — Pacing is presentation only. It must not be able to change which commands are accepted or in what order. A pacing bug degrades feel, never truth.

TRD-5.2.2 — The local match pump advances **exactly one** bot decision per tick and reschedules. It must not resolve a whole chain in one tick.

TRD-5.2.3 — reduce must not be called inside a React state updater. (v1.0 did this in useGame.dispatch; it is not idempotent under StrictMode double-invocation.) Commands are dispatched through a store action.

5.3 Local persistence

The client may persist **only** these, all versioned, all migrated, all read through try/catch:

Key	Contents	Format
cc.session	Access/refresh token pair, account id	v1
cc.settings	Sound, haptics, reduced motion, handedness, theme	v1
cc.localMatch	{format, version, seed, config, seats, commands[], savedAt}	v2, max age 7 days
cc.lastRoom	Last room code, for the rejoin affordance	v1

TRD-5.3.1 — **No balance, inventory, stat or rating is ever stored client-side as a source of truth.** They may be cached with a server-supplied etag and a TTL, and the cache is discarded on any conflict.

TRD-5.3.2 — Every persisted shape carries a format integer and has an explicit migration path. An unmigratable shape is discarded, not silently reinterpreted. v1.0's profile record had no version field at all.

5.4 Performance budgets

Budget	Target
Initial JS (gzipped, route-split)	≤ 180 KB
Total precached PWA payload	≤ 2.5 MB
Cold start to interactive menu (mid-range Android, 4G)	≤ 2.5 s p75
Time to first frame of the table after Start	≤ 400 ms
Sustained frame rate during card flight	60 fps; never below 45 fps p95
Main-thread task length	No task > 50 ms during a turn
Memory, 30-minute session	≤ 180 MB, no monotonic growth

TRD-5.4.1 — The splash screen ends when the asset prefetch ends, with a 400 ms floor and a 1500 ms backstop. It is not a fixed pad. (v1.0's fixed 1500 ms pad produced a 2,050 ms cold start against a page that had finished loading at 135 ms.)

TRD-5.4.2 — Card faces are rendered in CSS — one element, one face colour, one shard motif, one text glyph. No bitmap card art. This is both a performance and an accessibility decision.

6. Network protocol

6.1 Transport

- WebSocket over TLS. `wss://` in every non-local environment.
- JSON text frames. A binary framing is explicitly deferred; the message shape must not assume JSON forever.
- `PROTOCOL_VERSION` is sent in the connection handshake. A mismatch is refused with a specific code that the client turns into “please update”, not into a retry loop.

6.2 Connection lifecycle

1. Client obtains an access token over REST.
2. Client opens the socket with the token in the `Sec-WebSocket-Protocol` header or as a short-lived query ticket — **never** as a long-lived token in a URL that ends up in logs.
3. Server validates the token, binds the connection to an `accountId`, and replies `WELCOME { accountId, protocolVersion, serverTime }`.
4. Server sends application-level PING every 20 s; a connection with no PONG for 45 s is closed.

TRD-6.2.1 — **Identity comes from the token, never from the message.** v1.0's HELLO accepted a client-claimed `playerId`, and the only check was that the command's `playerId` matched the claim — which validated nothing. In v2.0 the server sets `session.accountId` from the verified token and ignores any identity field in any inbound message.

6.3 Client → server messages

Message	Notes
CREATE_ROOM { version, config?, maxSeats? }	maxSeats clamped server-side to SEAT_LIMIT; v1.0 trusted it verbatim Rate-limited; see §6.7
JOIN_ROOM { code }	
LEAVE_ROOM	
ADD_BOT { tier } / REMOVE_SEAT { playerId }	Host only — v1.0 omitted the host check on both Host only, lobby only Own seat only — v1.0 omitted this check Host only
SET_CONFIG { config } / SET_VERSION { version }	
READY { ready }	
START_GAME	
COMMAND { command, ackSeq }	Emote vocabulary is closed and server-validated Matchmaking
RESYNC { sinceSeq }	
CHAT { text } / EMOTE { emote }	
QUEUE_JOIN { version, mode } / QUEUE_LEAVE	

6.4 Server → client messages

WELCOME · ROOM · ROOM_CLOSED · SNAPSHOT · EVENTS · ACK · REJECTED · QUEUE_STATUS · CHAT · EMOTE · ERROR

ACK { commandId, seq } is **new in v2.0** and is the fix for the two protocol defects below.

6.5 Command acknowledgement — corrected semantics

v1.0 had two defects in this area. Both are specified out here.

Defect 1 — snapshot clears all pending. v1.0 treated any inbound SNAPSHOT as an acknowledgement and cleared the entire pending map. Snapshots are broadcast to every seat on every state change, so a snapshot caused by *another player's* move discarded this client's genuinely unacknowledged commands — defeating the reconnect replay the pending map exists for.

TRD-6.5.1 — A command is removed from pending only on an explicit ACK { commandId } or REJECTED { commandId } naming it. A SNAPSHOT never clears pending.

Defect 2 — racy stale-sequence rejection. v1.0's client sent ackSeq = lastSeq and the server rejected when ackSeq < state.seq. In any multi-seat match, another player moving between the client's last snapshot and the server's processing made a perfectly legal command fail with STALE_SEQUENCE.

TRD-6.5.2 — ackSeq is **advisory only**, used to compute the missed-event tail, and is never grounds for rejection. Replay protection is provided by the commandId idempotency set, which is what it was always for. A command is rejected only for a rule reason, an authorisation reason, or rate limiting.

6.6 Idempotency, sequencing and resync

Mechanism	Specification
Idempotency	Server keeps a set of accepted commandIds. A replay returns the original ACK; it is never re-applied and never rejected.
Sequence	state.seq is a monotonic count of accepted commands, attached to every EVENTS batch and every SNAPSHOT.
History ring	The last 400 {seq, events} batches are retained per room.
Resync	RESYNC { sinceSeq } returns a SNAPSHOT plus every event batch with seq > sinceSeq, each labelled with its original seq . v1.0 relabelled the whole replayed batch with the current seq.
Too far behind	If sinceSeq precedes the ring, the server sends SNAPSHOT with truncated: true. The client skips animation and hard-cuts to the snapshot instead of silently missing events.
Turn clock	resync must return the same decorated view as a normal snapshot, including the turn clock. v1.0's resync path omitted it.

6.7 Rate limits

Every inbound message class is limited. v1.0 limited only COMMAND.

Class	Limit
COMMAND	20 / second / account
JOIN_ROOM	10 / minute / account, 60 / minute / IP
CREATE_ROOM	6 / minute / account
ADD_BOT, REMOVE_SEAT, SET_CONFIG, SET_VERSION, READY	30 / minute / account
RESYNC	6 / minute / connection
CHAT	5 / 10 seconds, 240 characters
EMOTE	Cooldown per emote, per account
Connections	4 concurrent / account, 20 / IP

TRD-6.7.1 — Frame limits: maximum inbound message **64 KB**; maximum accumulated fragment buffer **256 KB**; permessage-deflate enabled with a context-takeover limit. v1.0's decoder accepted frames up to ~4 GiB with an unbounded fragment buffer and ignored socket.write backpressure — a straightforward memory denial of service.

6.8 Command handling — bots

TRD-6.8.1 — Bot commands go through the **same** submit path as human commands, including duplicate detection and invariant assertion. v1.0's submitBot bypassed rate limiting, duplicate detection and stale-sequence checks by taking a second reducer call site, which had to be kept in step by hand. Rate limiting is the one control that is scoped per-origin rather than disabled: bot moves are limited by the server's own scheduler, not by the account limiter.

7. Match service

7.1 Room model

A MatchRoom is transport-agnostic: no socket, no I/O. It emits RoomEvents that the gateway fans out. This is preserved from v1.0 and is why reconnect, duplicate-command and stale-sequence handling are unit-testable without opening a port.

Aspect	Specification
Status	Derived from engine state: no state or SETUP → LOBBY; PLAYING → PLAYING; otherwise ROUND_END
Seats	{ accountId, displayName, avatar, isBot, botTier?, ready, connected, isHost }
Max seats	min(clientRequested ?? SEAT_LIMIT.max, SEAT_LIMIT.max) = at most 6
Room codes	5 characters from a 32-symbol alphabet excluding 0 0 1 I; 33.5 M space; allocation is a Redis SET NX so codes are unique across the cluster , not per process
Code lifetime	Released when the room closes; not immediately reused
Join during play	Refused with MATCH_IN_PROGRESS unless the account owns a seat (reconnect)
Leave during play	Seat retained, marked disconnected
Host migration	On host leave, host moves to the first connected human seat
Room grace	90 s after the last connected human leaves, then closed

7.2 Authorisation matrix

Action	Who
SET_VERSION, SET_CONFIG, START_GAME ADD_BOT, REMOVE_SEAT	Host only Host only (v1.0 defect: any player could kick any seat, including the host)
READY COMMAND	Own seat only (v1.0 defect: any player could ready any seat) The seat bound to the authenticated connection only
CHAT, EMOTE	Any seated account

7.3 Turn clock

Parameter	Value
Default allowance	30,000 ms per decision, configurable, 0 disables
Owner	Server. Broadcast as { playerId, deadline, timeoutMs, now }
Clock subject	awaitingChoice.playerId ?? players[activePlayerIndex].id
Restart	On every state change — one allowance <i>per decision</i> , not per turn
Ordering	The clock is refreshed before snapshots are built, so the countdown is in the snapshot (a v1.0 bug that reached production)
Guard	An armed timer records armedAtSeq; on fire it no-ops if the seq moved or the subject changed

TRD-7.3.1 — Timeout auto-play, in this order, each command going through the normal submit:

1. A pending choice belonging to the subject: COLOR/DRAW_COLOR → first eligible colour; SWAP_HAND → first eligible target; otherwise CHOOSE_TARGET.
2. mayPass → END_TURN.
3. canDraw → DRAW_CARD then END_TURN.
4. Otherwise PLAY_CARD with the first legal card.

The loop breaks as soon as the seq stops advancing or the turn leaves the subject.

TRD-7.3.2 — Clients render the countdown from the server deadline and a measured clock offset. A client has no timer of its own and cannot give itself longer by lying about the time.

7.4 Bot scheduling

- One bot decision per tick, then reschedule. A recursion guard of 400 bails out.
- Delay: botDelayMs (default 700 ms; 0 in tests).
- When a **human** owes a Clash call, the bot delay becomes $\max(\text{botDelayMs}, \text{config.clashGraceMs})$ so the human gets the same window they would get in local play.
- Candidates: if awaitingChoice, only that bot; otherwise the bot owing a Clash call and the active-player bot.
- An illegal bot command is caught, logged with the view that produced it, and leaves state untouched. It is also a **test failure** in CI: the bot suite asserts zero illegal decisions across 10,000 simulated matches.

7.5 Match persistence and recovery

Aspect	Specification
Persisted shape	{ format, roomId, code, version, config, maxSeats, hostId, seed, matchSeed, roundCount, seats, rngCursor, checkpoint?: {seq, state}, commands: Command[], savedAt }
Storage	Redis for the live room; Postgres for the completed match record
Checkpoint	Every 40 accepted commands the reducer output is cloned into checkpoint, the command log is emptied, and the RNG cursor is captured with it
Write cadence	Coalesced, 400 ms debounce, flushed on shutdown
Restore	Replay is suppressed for bots, broadcasts, timers and saves; a command the engine now refuses stops the replay at the last agreed state with a warning rather than discarding the room
Format gate	An unreadable format refuses restore with UNREADABLE_SAVE rather than guessing

TRD-7.5.1 — Recovery is **not** the same thing as storage. The live room in Redis exists so a process restart does not kill a match. The permanent record of a completed match goes to Postgres at MATCH_ENDED and is never deleted by a reaper. v1.0 conflated these and therefore kept nothing.

7.6 Graceful shutdown

TRD-7.6.1 — On SIGTERM: stop accepting new rooms, flush every room to Redis, send ERROR { code: 'SERVER_DRAINING', retryAfterMs } and close each socket with code 1001, then exit. Clients treat 1001 + that code as an immediate reconnect, not a backoff. v1.0 called process.exit(0) and let every client discover the drop by timeout, so every deploy interrupted live matches.

8. Client reconnection

Parameter	Value
Backoff	$\min(500 \times 2^{\text{attempt}}, 8000)$ ms, plus $\pm 25\%$ jitter (v1.0 had no jitter, so a server restart produced a synchronised thundering herd)
Attempts	Unlimited while the app is foregrounded; paused when backgrounded
On open	Authenticate → JOIN_ROOM if a room is known → RESYNC { sinceSeq } → replay every entry still in pending under its original commandId
On REJECTED	Remove that command from pending, surface the reason in words, and — for any reason other than a rule reason — request a RESYNC
On token expiry mid-socket	Refresh over REST and re-authenticate in place, without dropping the socket

TRD-8.1 — Rejection reasons are mapped to player-facing sentences. A raw reason code must never be shown. v1.0 rendered the code itself as a toast for 1800 ms with no recovery action.

9. Configuration and environment

TRD-9.1 — Every environment variable is parsed and validated by a Zod schema at startup. A missing or malformed value **fails the process** with a message naming the variable. v1.0 used `Number(process.env.X)` with no validation, so a typo silently yielded NaN and disabled the turn timer.

Variable	Default	Purpose
NODE_ENV	—	Required
PORT	8787	HTTP/WS listener
DATABASE_URL	—	Required
REDIS_URL	—	Required
JWT_PUBLIC_KEY / JWT_PRIVATE_KEY	—	Required
CC_TURN_TIMEOUT_MS	30000	0 disables
CC_BOT_DELAY_MS	700	
CC_ROOM_GRACE_MS	90000	
CC_CHECKPOINT_EVERY	40	
CC_MAX_ROOMS_PER_NODE	500	Shed load above this
CC_ALLOWED_ORIGINS	—	Required; enforced on WS upgrade
OTEL_EXPORTER_OTLP_ENDPOINT	—	Telemetry

10. Security requirements

ID	Requirement
SEC-01	All traffic over TLS. HSTS on the web origin.
SEC-02	Access tokens ≤ 15 minutes; refresh tokens rotate on use, are stored hashed, and a reused refresh token revokes the whole family.
SEC-03	Passwords hashed with Argon2id. Password login is rate-limited and lockout-protected.
SEC-04	Origin is validated on WebSocket upgrade against CC_ALLOWED_ORIGINS. v1.0 performed no origin check at all.
SEC-05	A private hand is never present in any payload sent to another player. A PlayerView restricts cards to the viewer's own hand plus the top discard.
SEC-06	Every inbound message is schema-validated before any handler sees it.
SEC-07	Rate limits per \$6.7, enforced in Redis so they hold across nodes.
SEC-08	Room codes are not access control on their own for public rooms; a room carries a visibility flag and, for private rooms, an invite token embedded in the deep link.

ID	Requirement
SEC-09	Server logs never contain a token, a password, an email address, or a hand.
SEC-10	Dependencies are scanned on every build; a known critical advisory fails the build.
SEC-11	An anti-abuse surface: report a player, mute a player, and a moderation action that can rename or suspend an account.
SEC-12	Every accepted command is retained in the match record, giving a compact audit trail for dispute analysis.

11. Testing strategy

11.1 Layers and ownership

Layer	What it covers	Where
Unit	Every rule effect and validator, per ruleset	packages/game-engine/test
Property	Card conservation, no duplicate ids, turn reachability, determinism	properties.test.ts
Integration	Full rounds per version, 2–10 players	per-version suites
Network	Reconnect, duplicate command, stale sequence, late join, resync, checkpoint restore	apps/match
Protocol	Command stamping, acknowledgement, replay after a drop	apps/client/transport
Contract	Every client ↔ server message against the shared Zod schema, both directions	packages/shared
Edge cases	The complete §16.2 must-pass list	edgeCases.test.ts
API	REST surface, auth flows, authorisation matrix	apps/gateway
Data	Migrations up and down, ledger reconstruction	packages/db
Browser (e2e)	Playwright at three viewports: phone portrait, phone landscape, desktop	/e2e
Load	Synthetic concurrency against the match service	/tools/load

11.2 Must-pass edge cases (carried forward, all still required)

2-player Reverse · opening discard is an action or wild · draw pile empties while a penalty is pending · Clash called and missed around the same command boundary · player disconnects while a colour or target choice is pending · Mayhem stacking reaches a player who cannot answer · Mayhem hand hits exactly 25 after a draw · Flip occurs while animations are active · Wild Rush target is self or eliminated · Flex token consumption with simultaneous effects · late join after start · reconnect after missing three or more state updates.

New in v2.0: checkpoint restore preserves RNG continuity · a second device signing in takes over a seat cleanly · a node is killed mid-match and the match resumes elsewhere · a refresh token is replayed and the family is revoked · a balance is reconstructed from its ledger.

11.3 Determinism harness

TRD-11.3.1 — A property test runs N random seeds through a full match in each ruleset, twice, and asserts stats-FreeState equality and identical event sequences. Any code path that consumes randomness outside the injected RNG breaks this test, which is how `Math.random` is kept out of the engine.

11.4 CI gates

Every gate runs on every push. **All of them must execute** — this is Phase 0's entire purpose, because in v1.0 four of them had never run anywhere.

1. lint (ESLint, including the dependency-boundary rule)
2. format:check (Prettier)
3. typecheck — offline mode, against the hand-written types/ shims
4. typecheck:installed — **authoritative**, against real @types/*; the shims are narrower and will have accepted code the real definitions reject
5. test — unit, property, integration, network, protocol, contract
6. test:e2e — Playwright, three viewports
7. audit:brand — fails if the prior trademark appears in any shippable file
8. build:web, build:pwa, build:single — all three targets
9. Migration check — migrations apply cleanly to a fresh database and to the previous release's schema
10. Dependency and container vulnerability scan

11.5 Coverage expectations

- game-engine and game-content: 100% of branches in rule code. This is achievable because the code is pure.
- Match service room logic: ≥ 90% of branches.
- Client presentation: covered by e2e rather than by unit-test percentage.

12. Build and release pipeline

12.1 Build targets

Target	Output	Purpose
build:web	apps/client/dist	Web deployment; hashed, cacheable asset files
build:pwa	dist/pwa	Installable PWA and the Capacitor Android web assets; adds manifest, icon set, service worker
build:single	dist/color-clash.html	One self-contained offline document

Two asset registries, both generated from the same images, both exporting the same `AssetName` union and `ASSETS` shape so callers cannot tell them apart: `assets.gen.ts` (data URIs, for the single-file build) and `assets.files.ts` (file imports, for web and PWA, wired through a Vite alias).

TRD-12.1.1 — `assets.gen.ts` is generated and enormous. It is excluded from Prettier, ESLint and the branding audit, and must never be hand-edited.

TRD-12.1.2 — The three packaging failure modes that look fine in a build log and are blank on a device each have a unit test over a pure function: the document is genuinely self-contained; APK asset paths are relative, not root-absolute; the service worker precaches every emitted file.

12.2 Android release

Aspect	Specification
appId	com.geeksforgames.colorclash
webDir	dist/pwa
Splash	launchShowDuration: 600, background #070f20 (brand ink, so the hand-off to the web splash is invisible), CENTER_CROP, no spinner
compileSdk / targetSdk	Pinned as explicit constants in <code>tools/android-release.ts</code> — currently 35 — rather than inherited from Capacitor, so a dependency bump cannot silently drop below Play's annually-raised minimum
Signing	Four encrypted CI secrets: keystore base64, keystore password, key alias, key password. The workflow refuses to start if one is missing.

Aspect	Specification
Verification	apksigner and jarsigner verify the outputs are genuinely signed rather than assuming it
Artifacts	A signed .aab for Play and a matching signed .apk for sideload testing
versionCode	The CI run number — monotonic, never repeated
Key custody	Play App Signing enabled, so the generated key is an <i>upload</i> key and can be reset if lost. Keystore validity ≥ 2033 .

12.3 Service deployment

Aspect	Specification
Packaging	One container image per service, built once and promoted across environments
Environments	dev $\rightarrow$ staging $\rightarrow$ production, same images, different config
Migrations	Applied before the new version starts; every migration must be backward-compatible with the running version (expand, migrate, contract)
Rollout	Rolling, with SIGTERM draining per §7.6. Match nodes drain by refusing new rooms and waiting for existing rooms to end or checkpoint.
Rollback	Previous image plus a forward-only compensating migration. A migration that cannot be rolled forward safely is not merged.
Static client	Served from a CDN, not from the Node process. v1.0 read every file synchronously per request with no cache headers, which is a dev-box behaviour.

13. Observability

ID	Requirement
OBS-01	/healthz (liveness) and /readyz (dependencies reachable) on every service. v1.0 had no endpoints at all.
OBS-02	Structured JSON logs with a correlation id that follows a request from REST through the socket into the match room.
OBS-03	Metrics: active connections, active rooms, rooms per node, commands/second, command latency histogram, rejection rate by reason, queue depth and wait time, reconnect success rate, bot backfill rate.
OBS-04	Distributed traces on the REST surface and on room lifecycle transitions.
OBS-05	Alarms. Sev-1: any desync_detected; invariant assertion failure; readiness failure. Sev-2: rejection rate above baseline; matchmaking p99 above SLA; ledger reconstruction mismatch.
OBS-06	Client crash and error reporting with source maps, sampled, with no personal data in the payload.
OBS-07	A per-match diagnostic bundle — seed, config, command log, final state — retrievable by match id for dispute analysis.

14. Migration from v1.0

Item	Decision
Engine, content, AI, test-fixtures packages	Ported unmodified . Their existing tests are the acceptance gate.

Item	Decision
shared types	Ported, then extended with the new protocol messages and account types.
Client screens	Rewritten against the new routing and state layers, reusing the CSS and the presentation logic.
Server	Rewritten. Only MatchRoom's transport-agnostic design and the room lifecycle survive as a design, not as code.
Local localStorage saved match	Readable in v2.0 (format: 2) so an in-progress local match survives the update.
Local profile (name)	Name carried into the new guest account.
Local balances (12,450 coins / 860 gems)	Not migrated. They were a placeholder, never earned.
RULE_FLAGS.md	Carried forward verbatim, including RF-014's closure record.
BRANDING.md asset licence table	Carried forward and remains the gate for any new asset.

15. Definition of done

A change is done when:

1. It compiles under both typecheck modes.
2. Every CI gate passes.
3. No duplicate implementation of a rule exists.
4. UI state is derived from engine state; no rule logic entered a component.
5. Where a network path is involved, it is validated server-side.
6. Any ambiguity is explicit, documented in RULE_FLAGS.md, and pinned by a test.
7. The feature can be demonstrated from a clean start on a real device.
8. Rule logic and its tests landed in the same change.